

Documentación del proyecto AgenteApis

Descripción de **cada archivo** del agente: qué es y qué hace. Todo el código del agente vive en la carpeta `agent/`.

Archivos Python (`agent/`)

`main.py`

Programa que ejecutas con `python main.py`. Lee por consola lo que quieres que la API haga, llama a la lógica de generación, comprueba que el código devuelto parezca una app FastAPI y, si pasa las comprobaciones, guarda el resultado en disco. También muestra mensajes de error si falla el modelo o la validación.

`prompts.py`

Contiene el texto fijo que se envía al modelo (instrucciones, formato de salida) y la función que arma la lista de mensajes del chat a partir de lo que escribió el usuario. Sirve para cambiar cómo se comporta el agente sin tocar el resto del código.

`llm_client.py`

Conecta con la API del LLM (compatible con OpenAI). Carga opciones desde `agent/.env` y del sistema (`OPENAI_API_KEY`, etc.), envía los mensajes y devuelve el texto de la respuesta. Si algo falla (clave ausente, red, respuesta vacía), lanza `LLMError` con un mensaje entendible.

`generator.py`

Junta el requerimiento del usuario con los mensajes de `prompts.py`, pide la respuesta a `llm_client.py` y deja el código listo para guardar (por ejemplo quitando bloques markdown si el modelo los añade). Incluye una función que comprueba que el texto contenga `FastAPI()` antes de aceptarlo como API generada.

`file_writer.py`

Escribe el código generado en `agent/generated_api/main.py`. Crea la carpeta `generated_api` si no existe y, la primera vez, un `__init__.py` vacío para que puedas importar el módulo con Uvicorn sin problemas.

Archivo generado (salida del agente)

`agent/generated_api/main.py`

No es código fuente del agente: es el **resultado** que el modelo produce y el agente guarda. Suele ser una API FastAPI con modelos Pydantic, datos en memoria y rutas CRUD. Lo ejecutas aparte con Uvicorn cuando quieras probar la API.

Otros archivos en `agent/` (configuración y ayuda)

Archivo	Qué hace
<code>requirements.txt</code>	Lista de paquetes Python que hay que instalar con pip.
<code>.env</code>	Tus claves y opciones locales (por ejemplo <code>OPENAI_API_KEY</code>). No conviene subirlo a un repositorio público con secretos dentro.
<code>README.md</code>	Instrucciones breves de instalación y ejecución del agente.

Uso del modelo LLM

El agente no embebe un modelo local: **llama por red** a un servicio compatible con la API de OpenAI, usando el paquete oficial `openai` y el endpoint **Chat Completions** (`client.chat.completions.create`).

Qué se envía: una conversación en formato chat con dos mensajes: uno de sistema (las reglas fijas definidas en `prompts.py`) y uno de usuario (tu descripción de la API). El modelo debe devolver **solo texto**; ese texto se trata como código Python de la API.

Qué modelo se usa: por defecto el código usa el identificador `gpt-4o-mini`. Puedes cambiarlo sin tocar Python poniendo en `.env` (o en el sistema) la variable `OPENAI_MODEL` con otro nombre de modelo que acepte tu proveedor (por ejemplo otro modelo de OpenAI o el que exponga tu `OPENAI_BASE_URL`).

Otros ajustes relevantes:

- **OPENAI_API_KEY:** obligatoria para autenticarte contra el proveedor (en la práctica siempre hace falta con OpenAI).
- **OPENAI_BASE_URL:** opcional. Si la defines, el mismo cliente hablará con esa URL base (útil para proveedores compatibles con la API de OpenAI, no solo el dominio oficial).
- **Temperatura:** en `llm_client.py` la petición usa temperatura **0.2** (respuestas algo más estables y repetibles que con valores altos).

La respuesta que se usa es el **contenido del primer mensaje del asistente** en la respuesta de la API. Si viene vacío o falla la llamada, el agente muestra un error (`LLMError`).